

Skeleton post processing

To allow for the machine learning algorithm to read user movements, skeleton post processing was proposed by the client.

Skeleton post processing is a process of simplifying image or video footage to find key points of objects to allow for easy interpretation for programs. The created program was made to clean up and simplify video footage of a person to allow for easier interpretation of a person's movements by a deep learning algorithm.

Libraries used:

- Opencv: provides computer vision and image processing (<https://www.geeksforgeeks.org/python/python-play-a-video-using-opencv/>)
- Ultralytics YOLO: YOLO is a real object detection algorithm that is used to identify objects in video and pictures. (<https://docs.ultralytics.com/>)
- Numpy: python library for fast numerical computations

Video input

The current version of the program takes a predefined file input using cv2.VideoCapture from the opencv library. This will simply take the video and allow python to access it.

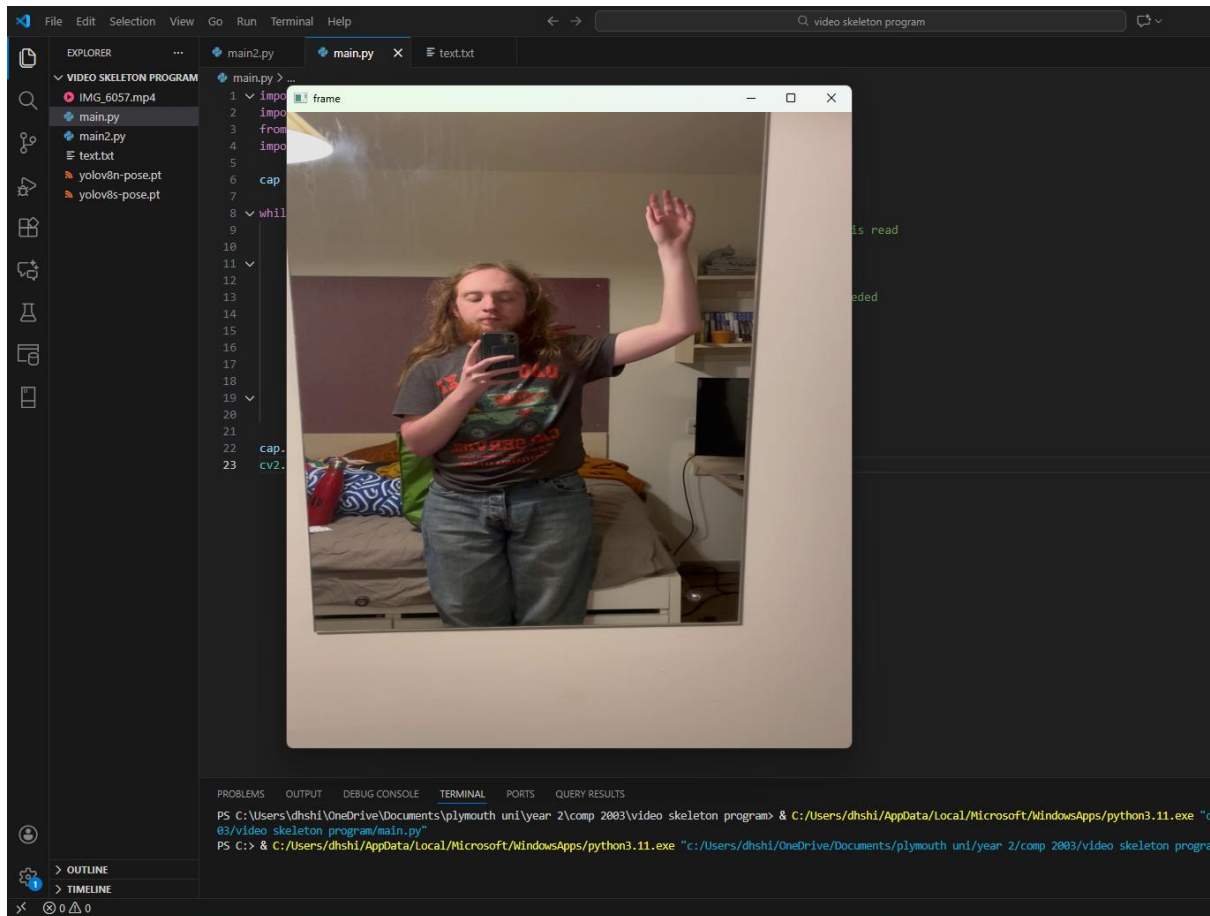
```
cap = cv2.VideoCapture('IMG_6057.mp4') #read the video
```

It is then output one frame at a time in a while loop.

```
8  while True:
9      ret, frame = cap.read()          #reads the video, ret is bool for if the video is read
10
11     if not ret:                      #if the video is not read try again
12         cap = cv2.VideoCapture('IMG_6057.mp4')
13         continue                    #this will loop the video forever, remove if needed
14
15     frame = cv2.resize(frame, (640,720)) #size of video
16
17
18     cv2.imshow('frame', frame) #show video
19     if cv2.waitKey(1) & 0xFF == ord('q'): #'q' pressed to end the loop
20         break
21
22     cap.release()                    #release system resources
23     cv2.destroyAllWindows()          #close the video
```

The video will loop indefinitely so an output has been added. Pressing q will end the program.

When the code is run, a window will open with the input video/ picture displayed on it.



Adding the model

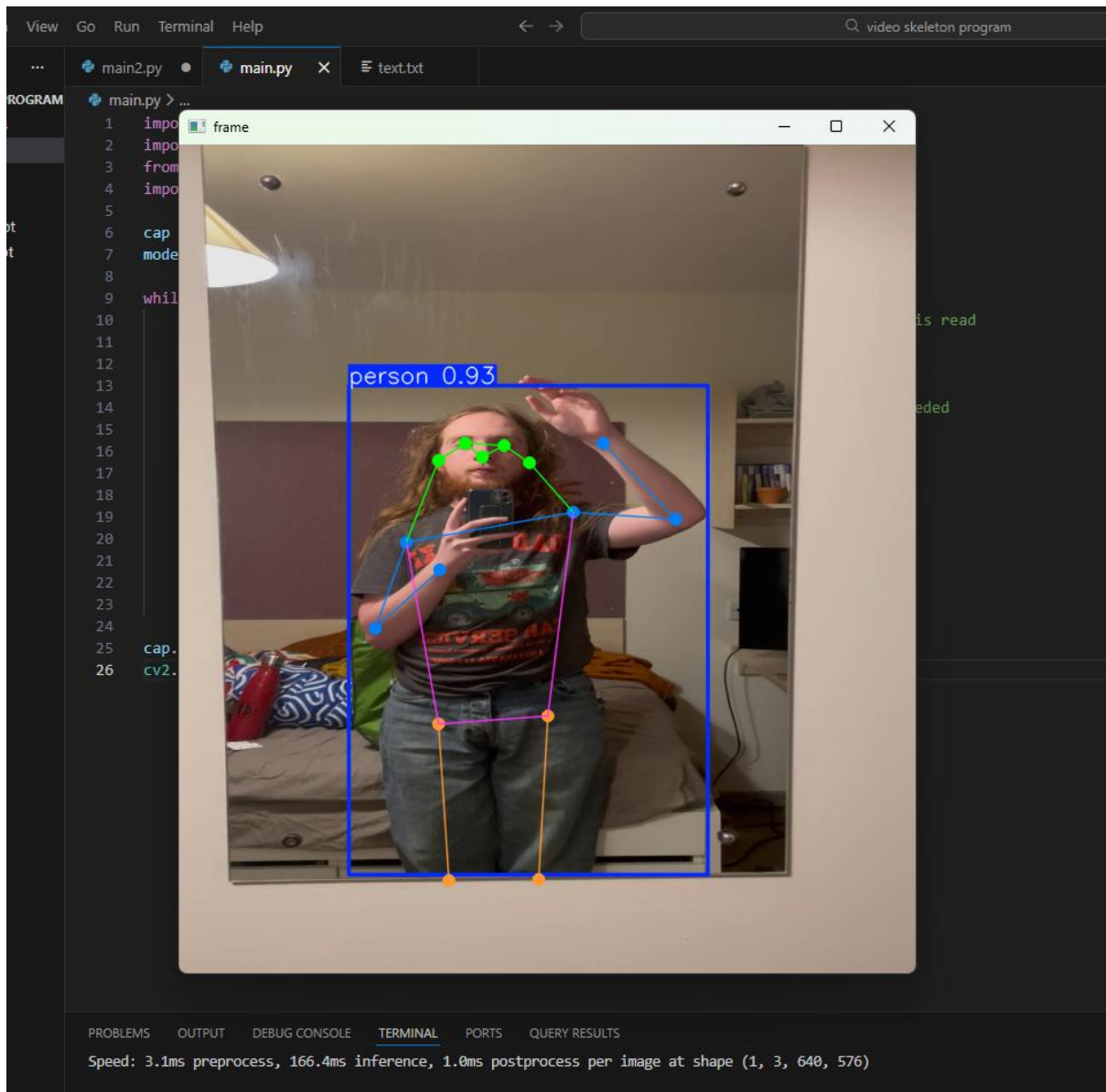
To add the model of YOLO, you simply add this line:

```
model = YOLO('yolov8n-pose.pt') #YOLO model for reading footage
```

The text in the brackets is the name of the version of the yolo model you want to use. Then in the while loop add these 2 lines:

```
res = model(frame) #skeleton of the current frame
frame = res[0].plot() #plot the skeleton on the original video
```

The first line is using the model to read the current frame of the video and find the key points of the person in the video. The second line is plotting them over the original video.



Each dot on the skeleton is key point being tracked. Each key point is allocated a number based on what it is tracking: (reference)

Number	Key point
0	Nose
1	Left eye
2	Right eye
3	Left ear
4	Right ear
5	Left shoulder
6	Right shoulder
7	Left elbow
8	Right elbow
9	Left wrist
10	Right wrist
11	Left hip
12	Right hip

13	Left knee
14	Right knee
15	Left ankle
16	Right ankle

Table 1: indexes of key points and what they track

Removing the background footage

Next part as to plot the key points on to a blank image. This will improve the readability of the key points.

```
width, height = frame.shape[:2]
blank = np.zeros((width, height, 3), dtype=np.uint8) #create blank image to make skeleton easier to read
```

This code creates a blank image background for the skeleton to be placed over.

```
for keypoints in res[0].keypoints.data: #get the keypoints of the
    keypoints = keypoints.cpu().numpy()

    for i, keypoint in enumerate(keypoints): #for each keypoint
        x,y,confidence = keypoint
        if confidence > 0.7: #plot the keypoints
            cv2.circle(blank, (int(x), int(y)), radius=5,
                color=(255,0,0), thickness=1) #keypoint marked with circle

            cv2.putText(blank, f'{i}', (int(x), int(y)-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255,255,255), 1,cv2.LINE_AA) #keypoints named with a number
```

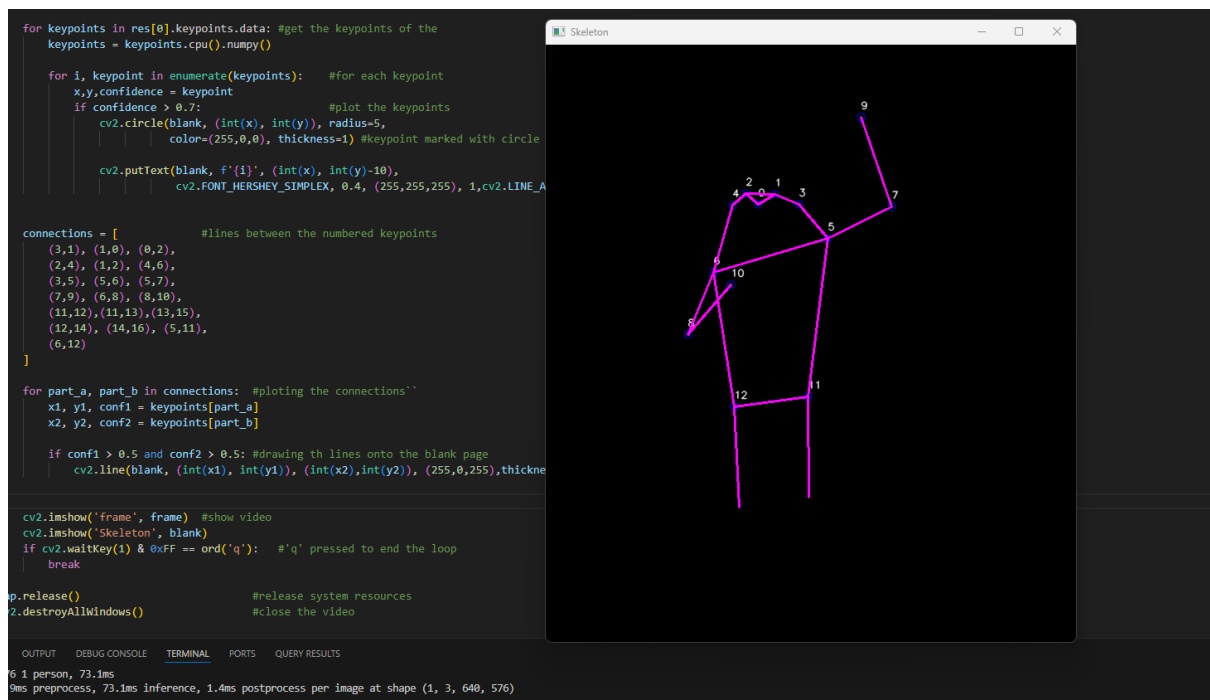
This code gets all of the key points and plots them as circles onto the blank image. It will also annotate each point with its respective number (see table 1).

```
connections = [ #lines between the numbered keypoints
    (3,1), (1,0), (0,2),
    (2,4), (1,2), (4,6),
    (3,5), (5,6), (5,7),
    (7,9), (6,8), (8,10),
    (11,12), (11,13), (13,15),
    (12,14), (14,16), (5,11),
    (6,12)
]

for part_a, part_b in connections: #plotting the connections``
    x1, y1, conf1 = keypoints[part_a]
    x2, y2, conf2 = keypoints[part_b]

    if conf1 > 0.5 and conf2 > 0.5: #drawing th lines onto the blank page
        cv2.line(blank, (int(x1), int(y1)), (int(x2),int(y2)), (255,0,255),thickness=2)
```

This code points out all of the connecting points with the connections array. This will be used to connect all of the key point together to create that outline from the original process. It will then get the coordinates of the points and draw a line between the 2 of them.



References:

- <https://docs.ultralytics.com/tasks/pose/> - pose estimation documentation